

プログラミング入門2

第4回

配列
for文
変数宣言
初期化

篠埜 功

今日の内容

- 配列
- for文
- 変数宣言
- 初期化

配列とは、同じ型のデータを格納する箱(領域)を一系列に並べて添え字でアクセスできるようにしたもの。

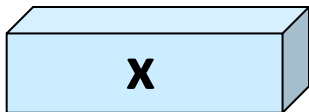
配列の宣言

- 3人分の点数を格納するint型の変数を用意したい場合

int型の変数の宣言

型名 変数名

int x ;

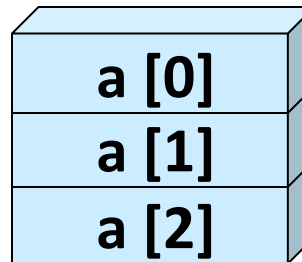


int型のデータの格納場所が1つ用意される。

int型を要素とする配列の宣言

要素の型名 変数名 [要素数]

int a [3] ;



a[0] ~ a[2]までの3個のint型のデータの格納場所が用意される。

[]内に書く番号(添え字という)は0から始まる。

配列の各要素へのアクセス

変数の場合の例

```
int x;  
x = 3;  
printf ("%d¥n", x + 2);
```

配列の場合の例

```
int x [10];  
x[0] = 3;  
printf ("%d¥n", x[0] + 2);
```

配列の各要素は、x[0]のように、配列名に続けて添え字を[]内に入れて表す。添え字は数字でなくても変数などの式でも構わない。ポインタの回にもう一度説明する。

変数の場合と同様に、配列の各要素がint型やdouble型の場合には、そこへ値を代入したり、そこに格納されている値を参照したりできる。

変数宣言の記法について(1)

これまで変数宣言は1つずつ書いてきたが、同じ型の変数はまとめて宣言することができる。

(例)

```
int x;
```

```
int y;
```

```
int z;
```

```
double a;
```

```
double b;
```

は、

```
int x,y,z;
```

```
double a, b;
```

のように、コンマで区切って一度に宣言してよい。

変数宣言の記法について(2)

配列の宣言もまとめて書いてよい。

(例)

```
int x;  
int a[3];  
int b[10];  
double y;  
double c[20];
```

は、

```
int x, a[3], b[10];  
double y, c[20];
```

のようにコンマで区切って一度に宣言してよい。

変数宣言と初期化

変数宣言時に、変数の初期値を書くことができる。

(例)

```
int x;
```

```
int y;
```

```
x=3;
```

```
y=10;
```

は、

```
int x=3, y=10;
```

のようにまとめて書いてよい。

変数宣言時に初期化をすることによって、その変数の値を代入前に参照するという状況が起こらなくなる。

配列の初期化(1)

変数と同様、配列も宣言時に初期化できる。
宣言時に、右辺に中括弧で囲んで値を並べる。

例えば、

```
int a[3];  
a[0] = 10;  
a[1] = 5;  
a[2] = 7;
```

は、

```
int a[3] = {10, 5, 7};
```

のようにまとめて書くことができる。

配列の要素数について

配列を宣言する際、要素数は定数にしてください(教科書 p. 117)。例えば、

```
void f (int n) {  
    int a[n];  
    ....  
}
```

のような配列の宣言(可変長配列)は(1990年のISO規格(C89)では)許されていない。

ただし、1999年のISO規格(C99)では定数でなくてもよくなったので、上記のような宣言は許された。

2011年のISO規格(C11)では、可変長配列はオプションとなった(コンパイラは可変長配列に対応してもしなくても良い)。

つまり、可変長配列は使わないようにするのが良い。

配列の初期化(2)

配列の初期化において、右辺の要素数が少ないときは、足りない部分は0 (double型の場合は0.0) で初期化される。たとえば、

```
int a[3] = {10, 5};
```

のように書くと、a[0]が10, a[1]が5, a[2]が0で初期化される。

配列の初期化(3)

配列の初期化において、右辺に初期化子がある場合、要素数を省略できる。例えば、

```
int a [ ] = {10, 5, 7};
```

のように書くと、

```
int a [3] = {10, 5, 7};
```

と書いたのと同じ意味になる。

例(打ち込んで確認)

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int a[3] = {10,5};
```

a[0]が10, a[1]が5, a[2]が0で初期化される。

```
    int b[3] = {0};
```

b[0], b[1], b[2]が0で初期化される。

```
    int c[ ] = {3,4,5};
```

c[0]が3, c[1]が4, c[2]が5で初期化される。

```
    int i=0;
```

```
    while (i<3) {
```

```
        printf ("a[%d] = %d¥n", i, a[i]);
```

```
        printf ("b[%d] = %d¥n", i, b[i]);
```

```
        printf ("c[%d] = %d¥n", i, c[i]);
```

```
        i=i+1;
```

```
    }
```

```
    return 0;
```

```
}
```

for文

これまでは繰り返しのための構文としてはwhile文を使用していた。繰り返し構文は配列を扱う場合によく使われる。

配列の処理の典型的な形:

```
i = 0;
while (条件式) {
    ... 配列に関する計算 ...
    i = i + 1;
}
```

このような形のプログラムを見やすく書くための構文としてfor文がある。

for文の例（左のプログラムを 打ち込んで確認）

```
#include <stdio.h>
int main (void) {
    int i, a[5]={1,2,3,4,5};
    for (i=0; i<5; i=i+1)
        printf ("a[%d]=%d¥n", i, a[i]);
    return 0;
}
```

```
#include <stdio.h>
int main (void) {
    int i, a[5]={1,2,3,4,5};
    i=0;
    while (i<5) {
        printf ("a[%d]=%d¥n", i, a[i]);
        i=i+1;
    }
    return 0;
}
```

左と右のプログラムは同じ意味である。

for文の構文(基本形)

for文の構文

for (式; 式; 式) 文

for文 $\text{for}(e_1; e_2; e_3) s$ の意味

```
 $e_1$ ;  
while ( $e_2$ ) {  
     $s$   
     $e_3$ ;  
}
```

と同じ意味である。

(注意) 1999年のISO規格 (C99)においては e_1 のところに変数宣言(有効範囲はfor文内部のみ)が書けるようにfor文の定義が拡張された(教科書p.94)。
 e_1 のところが変数宣言の場合は、左の置き換えはできない(変数の有効範囲が変わってしまうので)。

例

```
#include <stdio.h>
int main (void) {
    int i, sum=0, a[5]={1,2,3,4,5};
    for (i=0; i<5; i=i+1) {
        printf ("a[%d]=%d¥n", i, a[i]);
        sum = sum + a[i];
    }
    printf ("sum=%d¥n", sum);
    return 0;
}
```

配列aの要素の
和を表示するプ
ログラムである。

この例では、for文
の本体(赤字部分)
が複合文である。

while文を使った場合

```
#include <stdio.h>
int main (void) {
    int i, sum=0, a[5]={1,2,3,4,5};
    i=0;
    while (i<5) {
        printf ("a[%d]=%d¥n", i, a[i]);
        sum = sum + a[i];
    }
    i=i+1;
}
printf ("sum=%d¥n", sum);
return 0;
}
```

前ページのプログラムをさきほどの説明の通りwhile文で置き換えると左のプログラムになる。

(注)赤字の複合文の中括弧 {} を取り除いても同じ意味である。

for文の構文

さきほどfor文の構文を以下のように定義したが、括弧内の3つの式はそれぞれ省略可能である。

for (式; 式; 式) 文

1番目の式がない場合は、繰り返しの実行前に何もしないということである。

3番目の式がない場合は、各繰り返しにおいて、for文の本体の実行後、何もしないということである。

2番目の式がない場合は、繰り返しの条件が常に真という意味である。

例（左のプログラムを 打ち込んで確認）

```
#include <stdio.h>
int main (void) {
    for (;;)
        printf ("hello¥n");
    return 0;
}
```

```
#include <stdio.h>
int main (void) {
    while (1)
        printf ("hello¥n");
    return 0;
}
```

式を3つとも省略すると、while(1)で置き換えたプログラムと同じ意味である。
（無限にhelloと出力し続けるので、Ctrl-Cで終了させる。）

配列のコピー

配列のコピーを行う場合は、各要素をコピーする必要がある。

間違った例

```
int a[3] = {10, 5, 7};  
int b[3];  
b=a;
```

b=aの代入式はコンパイル時にエラーになる。
詳しくはポインタの回に説明する。

配列のコピー(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int a[5] = {3,4,5,6,7};
    int b[5];
    int i;
    for (i=0; i<5; i=i+1)
        b[i] = a[i];
    for (i=0; i<5; i=i+1)
        printf ("a[%d]=%d, b[%d]=%d¥n", i, a[i], i, b[i]);
    return 0;
}
```

b[i] = a[i]の代入式によって各要素ごとに代入を行っている。

多次元配列

配列の要素は配列でもよい。

例えば、int型を要素にもつ長さ3の配列を要素にもつ長さ2の配列は、

```
int a [2] [3];
```

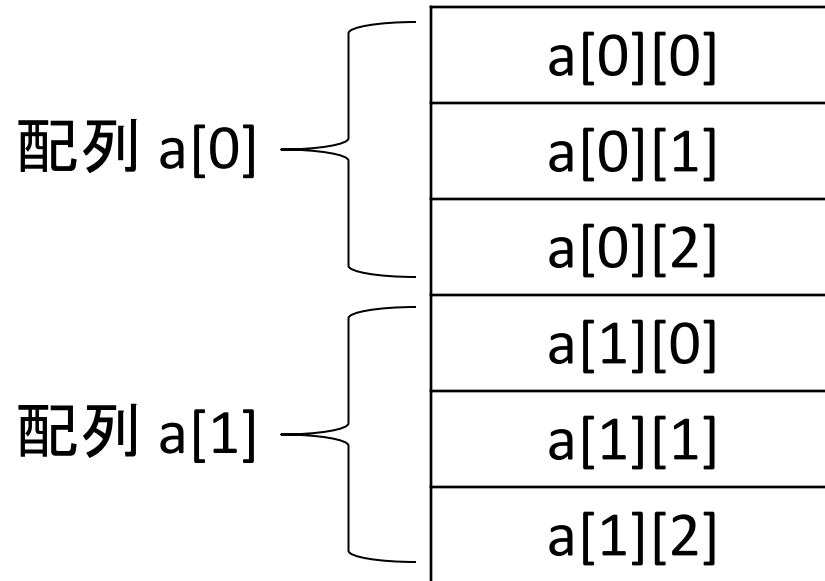
のように宣言する。これは2次元配列である。(3次元以上も同様に宣言できる。2次元以上の配列を多次元配列と呼ぶ。)

各要素は、a[0][1]のように、[]を並べて書くことによって表す。これは(a[0])[1]を括弧を省略して書いたものである。a[0]はaの0番目の要素(int型を要素にもつ長さ3の配列)を表し、a[0][1]は、配列a[0]の1番目の要素を表す。

多次元配列のメモリ上での配置

```
int a [2] [3];
```

のように宣言された2次元配列の各要素は以下のようにメモリ上に配置される。



多次元配列の例（打ち込んで確認）

```
/* 2*3の配列の各要素に1を代入し、各要素の値を表示 */
#include <stdio.h>
int main (void) {
    int i, j;
    int a[2][3];
    for (i=0; i < 2; i=i+1)
        for (j=0; j < 3; j=j+1)
            a[i][j] = 1;
    for (i=0; i < 2; i=i+1)
        for (j=0; j < 3; j=j+1)
            printf ("a[%d][%d] = %d¥n", i, j, a[i][j] );
    return 0;
}
```

多次元配列の初期化

1次元配列と同様に初期化できる。

(例)

```
int a [2][3] = { {1,2,3}, {4,5,6} };
```

のように宣言、初期化すると、配列 a[0] が{1,2,3}で初期化され、配列 a[1] が{4,5,6}で初期化される。

プログラム例 (打ち込んで確認)

```
/* 初期化した値を表示して確認 */  
#include <stdio.h>  
int main (void) {  
    int a[2][3] = { {1,2,3}, {4,5,6} };  
    int i, j;  
    for (i=0; i < 2; i=i+1)  
        for (j=0; j < 3; j=j+1)  
            printf ("a[%d][%d] = %d¥n", i, j, a[i][j]);  
    return 0;  
}
```

よくある間違い(右のプログラムを打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int i, x, y, z, a[4];
    for (i=0; i<4; i=i+1)
        a[i]=1;
    return 0;
}
```

問題ないプログラム

```
#include <stdio.h>
int main (void) {
    int i, x, y, z, a[4];
    for (i=0; i<=4; i=i+1)
        a[i]=1;
    return 0;
}
```

問題のあるプログラム

左のプログラムはa[0]からa[3]に1を代入します。右のプログラムはa[0]からa[3]に1を代入した後でa[4]に1を代入しようとしています。配列aの長さは4なので、a[4]は配列の範囲(境界)をはみ出しています。ほとんどの処理系でコンパイルが通り、実行時にa[4]に1が代入されてしまいます。a[4]の場所は別の変数や関数の戻り番地など、他の目的で使われている場所かもしれません。学情のgccで実行すると右のプログラムはa[4]の場所がiの場所になっており、a[4]に1を代入するとiの値が1になり、無限ループになります。最適化によっても変わります。gcc -O1 test.cなどとすると無限ループにならないかもしれません。

基本課題1

情報工学科のある科目のある年の成績一覧(seiseki.txt)と、点数を全員分表示するプログラム(a.c)を講義用web pageに置いてある。a.cを参考にして、この成績一覧の中で80点以上の人数を以下のように表示するプログラムを書け。

[実行例]

```
% ./a.out < seiseki.txt  
80点以上的人是81人です。
```

(注意) a.cとseiseki.txtは各自、講義用web pageからダウンロードしてください。実行方法は上の実行例と同じです。seiseki.txtは実行ファイル(a.out)と同じディレクトリに置いてください。

基本課題2

基本課題1の成績一覧について、平均点を以下のように小数点以下2桁まで表示するプログラムを書け。

[実行例]

```
% ./a.out < seiseki.txt
```

平均点は82.70点です。

平均点はdouble型で求め、変換指定子を%.2fにしてprintfで表示してください。%.2fは小数点以下2桁まで表示するという意味です。

(ヒント) 点数の合計をint型で計算し、人数で割ればよい。割り算の際に、分子か分母をキャスト演算子(double)を使って型変換すると、double型で平均点を得られる。

発展課題1

256*256の24bitのbmp画像を右90度回転させるプログラム (bmp.c)を講義用web pageに置いてある。これを参考にして、右180度回転させるプログラムを書け。(画像を裏返さずに180度回転させてください)

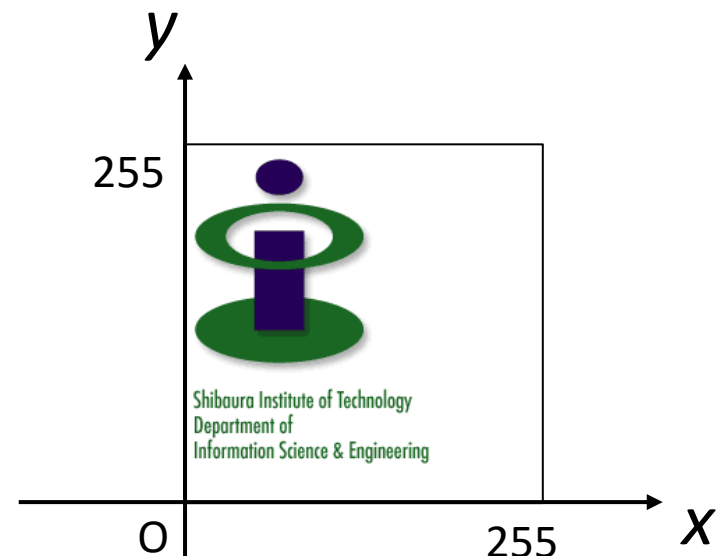
[実行例]

```
% ./a.out < joho.bmp > joho2.bmp
```

joho.bmpは講義用web pageに置いてあるので各自ダウンロードし、実行ファイル (a.out)と同じディレクトリに置いてください。

(注意) bmp.cは256*256の24bit以外のbmp画像では正常に動作しません

配列の境界を超えてアクセスしないように注意してプログラムを書いてください。



発展課題2

count.cは、romeo.txtの中で'a'が何回出現しているかを数えるプログラムである。これを参考にして、count.cに"my"の出現回数を数える機能を追加せよ。

[実行例]

```
% ./a.out < romeo.txt
```

'a'は8318回出現しました。

"my"は325回出現しました。

count.cとromeo.txtは講義用web pageに置いてあるので各自ダウンロードしてください。romeo.txtは実行ファイル(a.out)と同じディレクトリに置いてください。(romeo.txtは<https://gutenberg.org/files/1513/1513-0.txt>からダウンロードしたものです。)

配列の境界を超えてアクセスしないように注意してプログラムを書いてください。

発展課題3

2次元配列を3つ使って、2*2の行列の積を計算して表示するプログラムを書け。行列の各要素の値はint型とし、キーボードから読み込むようにせよ。ただし、2次元配列を使うことを条件とする。(3重ループを使うのが望ましいが、そうでなくても正解とする。)

[実行例]

行列aを入力してください:

a[0][0] = 1

a[0][1] = 2

a[1][0] = 3

a[1][1] = 4

行列bを入力してください:

b[0][0] = 1

b[0][1] = 1

b[1][0] = 1

b[1][1] = 1

行列a,bの積は

p[0][0] = 3

p[0][1] = 3

p[1][0] = 7

p[1][1] = 7

です。

発展課題4

2*2のint型の行列のn乗を計算し、表示するプログラムを書け。行列の値およびnはキーボードから読み込むようにせよ。ただし、2次元配列を使うことを条件とする。(行列の掛け算部分は3重ループで書くのが望ましいが、そうでなくても正解とする。)nは1以上とする。

[実行例]

行列aを入力してください:

a[0][0] = 1

a[0][1] = 2

a[1][0] = 3

a[1][1] = 4

何乗しますか: 3

行列aの3乗は

p[0][0] = 37

p[0][1] = 54

p[1][0] = 81

p[1][1] = 118

です。

(ヒント) 単位行列にaをn回掛ける形にすると分かりやすい。

発展課題5

キーボードから1以上65535以下の整数(int型)を受け取り、それを2進数の表記で画面に表示せよ。ただし、最上位桁は1になるように表示せよ。入力が上記の範囲に入っていることは仮定してよい。

(ヒント) 長さ16の配列を宣言し、下位から順に0または1を格納していき、最後に逆順に表示すればよい。

[実行例]

```
$ ./a.out
```

1から65535までの正の整数を入力して下さい: **10**

10を2進数で表記すると、1010です。

```
$ ./a.out
```

1から65535までの正の整数を入力して下さい: **10000**

10000を2進数で表記すると、10011100010000です。

```
$
```

参考課題1

キーボードから10人分の点数(100点満点)を受け取って長さ10の配列に格納し、最高点、最低点、平均点を計算して表示するプログラムを書け。平均点はdouble型で計算し、表示せよ。

[実行例]

点数を10人分入力してください:

1人目: 10

2人目: 50

3人目: 38

4人目: 80

5人目: 60

6人目: 90

7人目: 38

8人目: 100

9人目: 45

10人目: 87

最高点は100点, 最低点は10点, 平均点は59.800000点です。

参考課題1 解答例

```
#include <stdio.h>
int main (void) {
    int a[10];
    int i,max,min,sum=0;
    printf ("点数を10人分入力してください: ¥n");
    for (i=0; i<10; i=i+1) {
        printf("%d人目: ", i+1);
        scanf ("%d", &a[i]);
    }
    max = min = a[0];
    for (i=0; i<10; i=i+1) {
        sum = sum + a[i];
        if (max < a[i]) max = a[i];
        if (min > a[i]) min = a[i];
    }
    printf ("最高点は%d点, ", max);
    printf ("最低点は%d点, ", min);
    printf ("平均点は%f点です。¥n", (double)sum/10);
    return 0;
}
```

参考課題2

2次元配列を3つ使って、2*2の行列の和を計算して表示するプログラムを書け。行列の各要素の値はint型とし、キーボードから読み込むようにせよ。

[実行例]

行列aを入力してください:

a[0][0] = 1

a[0][1] = 2

a[1][0] = 3

a[1][1] = 4

行列bを入力してください:

b[0][0] = 1

b[0][1] = 1

b[1][0] = 1

b[1][1] = 1

行列a,bの和は

s[0][0] = 2

s[0][1] = 3

s[1][0] = 4

s[1][1] = 5

です。

参考課題2 解答例

```
#include <stdio.h>
int main (void){
    int a[2][2], b[2][2], s[2][2], i, j;
    printf("行列aを入力してください:¥n");
    for (i=0; i<2; i=i+1)
        for (j=0; j<2; j=j+1) {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    printf("行列bを入力してください:¥n");
    for (i=0; i<2; i=i+1)
        for (j=0; j<2; j=j+1) {
            printf("b[%d][%d] = ", i, j);
            scanf("%d", &b[i][j]);
        }
}
```

```
/* 続き */
for (i=0; i<2; i=i+1)
    for (j=0; j<2; j=j+1)
        s[i][j] = a[i][j] + b[i][j];
printf("行列a,bの和は¥n");
for (i=0; i<2; i=i+1)
    for (j=0; j<2; j=j+1)
        printf("s[%d][%d] = %d¥n", i, j, s[i][j]);
printf("です。¥n");
return 0;
}
```